



HCD · Historial Clínico Digital

Historia clínica soberana en Solana

Plan del proyecto. El paciente es dueño de su historia: los médicos verificados cargan estudios firmados y otros médicos solo los leen con su permiso, por tiempo limitado y con registro de cada acceso.

VERSIÓN

2.0 · 3 de octubre de 2026

EQUIPO

Misael, Matías, Maximiliano,
Franco, Rodrigo

REEMPLAZA A

Plan v1.0 (mismo día)

CIERRE

13/10/2026 · 03:59 (Argentina)

EVENTO

Superteam Argentina · Road to
Colosseum

RED

Solana devnet

Contenido

0	Cambios respecto de la v1	8	Cifrado y servicio de llaves
1	Resumen	9	Backend
2	Alcance del MVP	10	Frontend
3	Actores	11	Repositorios
4	Flujos	12	Riesgos principales
5	Principios de diseño	13	Plan de 10 días
6	Stack	14	Demo
7	Programa de Solana	15	Hoja de ruta

0 Cambios respecto de la v1

Esta versión incorpora los cambios aceptados por el equipo el 3/10/2026, a partir del relevamiento de Franco y las decisiones registradas en `docs/proyecto/decisiones.md`. El resto del documento ya está escrito con estos cambios aplicados.

Tema	v1 decía	v2
Quién carga estudios	Personal de la clínica	Solo el médico verificado (con matrícula), firmando con su propia wallet. Se elimina el rol "personal de clínica que carga".
Rol de la clínica	Emite estudios	No carga estudios. Avala qué médicos le pertenecen.
Estados del Record	Pending → Accepted / Rejected	Active al cargarse → Disputed (el paciente marca "no es mío") → Voided (el emisor lo anula y lo reemite).
Instrucciones del paciente sobre estudios	<code>accept_record</code> / <code>reject_record</code>	Solo <code>dispute_record</code> . El emisor tiene <code>void_record</code> .
<code>grant_access</code>	Exige Record Accepted	Exige Record Active .
Lectura del emisor	—	El médico emisor puede releer sus estudios sin permiso.
Edición	—	No existe ninguna instrucción que modifique un Record. El paciente solo lee.
Fee payer	Backend como fee payer	Igual, y además el backend arma la transacción , el usuario firma y el backend verifica byte a byte antes de co-firmar. Rate limit por usuario y organización, presupuesto diario, hot wallet con poco saldo y alerta.
Llaves del backend	—	Fee payer y autoridad <code>key_service</code> en keypairs distintos .

Tema	v1 decía	v2
Payer de cuentas	Signer separado del dueño	Igual. Si se cierran cuentas, el rent vuelve al sponsor (<code>rent_payer + close = rent_payer</code>).
Visor	Recalcula el hash	Obligatorio: si no coincide, muestra "Estudio alterado" y no descifra.
Login y wallet	Privy	Login con Supabase Auth; Privy solo para la wallet (no Cavos). RLS con <code>app_user</code> .
Por dónde empezar	Monorepo y programa en paralelo	Primero el programa Anchor: todo depende de su IDL.
Nombre	Pendiente	HCD · Historial Clínico Digital.
Estructura	Un monorepo con pnpm	Cuatro repos: <code>hcd</code> , <code>hcd_api</code> , <code>hcd_app</code> , <code>hcd_landing</code> (sección 11).
Backend	Express + Sequelize	NestJS + supabase-js , sin ORM.
Gestor de paquetes	pnpm	npm en todos los repos.
Validación de origen (caso Pepito)	—	El visor muestra el origen del estudio y los datos completos del emisor (sección 8).

1 Resumen

El paciente es dueño de su historia clínica. Los médicos verificados cargan estudios; otros médicos solo los leen si el paciente lo aprueba, por tiempo limitado y con registro de cada acceso. Solana es la fuente de verdad de permisos, prestadores habilitados y auditoría. **Los documentos médicos nunca tocan la cadena.**

Diferenciación: vencimiento real del acceso (servicio de llaves), un solo cliente primero e interoperabilidad HL7 FHIR (en el MVP, solo una exportación de ejemplo).

Plazo: cierre el 13/10/2026 a las 06:59 UTC, que son las **03:59 del 13/10 en Argentina**. En la práctica, entregar antes de la noche del 12/10.

2 Alcance del MVP

Dentro del MVP	Fuera del MVP (se documenta, no se construye)
Login con email o Google (Supabase Auth) y wallet embebida (Privy) para paciente, médico y admin	Cómputo confidencial con Arcium
Alta de clínicas y médicos; verificación manual por el equipo (admin)	Recuperación avanzada con Swig (MVP: recuperación de Privy)
Carga de estudios por el médico con el QR del paciente, cifrado en el navegador	App nativa con Solana Mobile (MVP: PWA)

Dentro del MVP	Fuera del MVP (se documenta, no se construye)
El paciente puede marcar un estudio como "no es mío"	Acceso de emergencia y contactos de confianza
Solicitud de acceso del médico y aprobación del paciente con vencimiento	Carga masiva con Merkle roots
Servicio de llaves que niega por defecto	Integración real con registros de matrícula y firma digital legal
Visor con marca de agua, verificación de hash, revocación y línea de tiempo de accesos	Borrado criptográfico automatizado (solo diseño)
Demo completa en Solana devnet	Mainnet, auditoría del programa, exportación FHIR completa

3 Actores

Actor	Rol	Cómo entra
Paciente	Dueño de su historia. Lee sus estudios, marca "no es mío", aprueba o revoca accesos.	PWA, Supabase Auth, wallet embebida de Privy
Médico emisor	Carga estudios firmados con su wallet. Puede releer los suyos.	Web, Supabase Auth, wallet de Privy, matrícula verificada por el admin
Médico lector	Pide acceso y lee mientras el permiso esté vigente.	Web, Supabase Auth, wallet de Privy, matrícula verificada
Clínica	Avala qué médicos le pertenecen. No carga estudios.	Panel web
Admin	Habilita clínicas y médicos. En el MVP, el equipo.	Wallet de autoridad del programa
Servicio de llaves	Único componente que entrega llaves, solo con permiso vigente.	Backend con keypair propio (autoridad key_service)

4 Flujos

A · Alta del paciente

Abre la PWA, entra con Supabase Auth (email o Google), Privy le crea la wallet embebida y se registra su perfil (`register_patient`). El DNI no se guarda: solo se usa para verificar a la persona en persona.

B · El médico carga un estudio

1. El paciente muestra un QR con su wallet pública y un código corto que vence en 2 minutos.
2. El médico verifica el DNI en persona y escanea el QR. El destino es la wallet, nunca una búsqueda por DNI.
3. El navegador genera una llave AES-256 por estudio (DEK) y cifra el archivo.
4. Sube el archivo cifrado con una URL firmada. La DEK va al servicio de llaves, que la envuelve y guarda solo la versión envuelta.
5. El médico firma `issue_record` con el hash del archivo cifrado. El Record queda **Active**.
6. El paciente recibe una notificación. Si no es suyo, lo marca con `dispute_record` y el emisor lo anula con `void_record`.

C · Un médico lee un estudio

1. El paciente comparte un código o QR. El médico pide acceso (off-chain, para evitar spam y costos).
2. El paciente ve quién pide (matrícula verificada) y qué estudios, y elige la duración (1 hora, 24 horas o 7 días). Firma `grant_access`.
3. El médico pide la llave. El servicio lee el Record y el permiso en Solana y comprueba que estén activos y sin vencer.
4. Entrega la llave y una URL firmada de 60 segundos, y registra el acceso con `log_access`.
5. El visor verifica el hash, descifra y muestra el estudio sin botón de descarga y con marca de agua (nombre, matrícula y fecha).
6. Al vencer el permiso no se entregan más llaves. El paciente puede revocar antes con `revoke_access`.

Si el paciente no aprueba, no existe el permiso y el servicio niega por defecto: la ausencia de permiso es el bloqueo.

Estados de un estudio (Record)



Active desde que se carga. Solo un Record Active puede compartirse con `grant_access`. Nadie puede editar un Record.

5 Principios de diseño

- **Nada médico en la cadena**, ni siquiera cifrado.
- **Negar por defecto**.
- **Una llave por estudio**: un permiso abre un estudio, no toda la historia.
- **El destino es una wallet**, no un DNI.
- **Aislamiento por organización**.
- **Almacenamiento borrable** (nada de Arweave o Irys).
- **Honestidad sobre los límites**: el servicio de llaves es un punto de confianza y un médico puede copiar lo que ya vio.

En Solana va solo lo que prueba algo: identidad, permisos, hashes, firmas y registro de accesos. La Ley 25.326 trata la salud como dato sensible, con derecho a rectificar y suprimir, y una cadena no borra.

6 Stack

Capa	Elección
Programa	Rust + Anchor 1.2, cliente con Codama y @solana/kits. En Windows, Anchor requiere WSL.
App y landing	Next.js 16 + React 19 + TypeScript 6 + Tailwind + shadcn/ui. Supabase Auth para el login, Privy para la wallet, WebCrypto para cifrar.
Backend	NestJS 12 + supabase-js (sin ORM) + Zod. Aislamiento entre organizaciones con RLS.
Datos	PostgreSQL y Storage en Supabase. Solana devnet con RPC de Helius y RPC público de respaldo.
Hosting	Vercel (app y landing), Railway o Render (API, a confirmar), Supabase (datos).
Paquetes	npm en todos los repos. Node.js 24.

El detalle con versiones, repo por repo, está en `docs/proyecto/stack.md`.

7 Programa de Solana

Cuentas (PDAs)

Cuenta	Semillas	Campos
Config	<code>["config"]</code>	admin, key_service, duración máxima de permisos
Provider	<code>["provider", authority]</code>	authority, tipo (clínica o médico), verified, organización
PatientProfile	<code>["patient", authority]</code>	authority, next_record_id, creado
Record	<code>["record", patient, record_id]</code>	patient, issuer, content_hash (SHA-256 del archivo cifrado), storage_ref, status (Active , Disputed , Voided), created_at. Propuestos: <code>rent_payer</code> , <code>supersedes</code>
AccessGrant	<code>["grant", record, doctor]</code>	patient, doctor, record, expires_at, status (Active o Revoked), access_count. Propuesto: <code>rent_payer</code>

Instrucciones

Instrucción	Quién firma	Qué hace
<code>initialize_config</code>	Admin	Configuración global y autoridad del servicio de llaves
<code>register_provider / set_provider_verified</code>	Prestador / Admin	Alta del prestador; solo el admin lo verifica
<code>register_patient</code>	Paciente	Crea el perfil con contador de estudios
<code>issue_record</code>	Médico verificado	Crea el Record en estado Active
<code>dispute_record</code>	Paciente	Active → Disputed ("no es mío")
<code>void_record</code>	Médico emisor	Disputed → Voided; se reemite con un Record nuevo

Instrucción	Quién firma	Qué hace
<code>grant_access</code>	Paciente	Crea el permiso. Exige Record Active, vencimiento futuro y menor al máximo
<code>revoke_access</code>	Paciente	Pasa el permiso a Revoked. No se cierra la cuenta, para conservar auditoría
<code>log_access</code>	Servicio de llaves	Incrementa <code>access_count</code> y emite evento. Exige permiso Active y vigente

Eventos: RecordIssued, RecordDisputed, RecordVoided, AccessGranted, AccessRevoked, AccessLogged. El contador `access_count` es la fuente de verdad (los logs pueden truncarse).

Seguridad: validar signers, semillas y bump con restricciones de Anchor; comparar el vencimiento contra `clock`, nunca contra un valor del cliente; no guardar nombres, diagnósticos, DNI ni rutas legibles; probar casos negativos (médico sin permiso, permiso vencido o revocado, estudio en disputa, prestador sin verificar); solo devnet sin auditoría externa.

Quién paga la red

- Paciente y médico **nunca necesitan SOL**. El backend es fee payer y el costo lo absorbe el cliente B2B.
- El backend arma la transacción, el usuario firma y el backend verifica byte a byte antes de co-firmar.
- Rate limit por usuario y organización, presupuesto diario, hot wallet con poco saldo y alerta.
- Fee payer y `key_service` son keypairs distintos: robar el primero cuesta SOL; robar el segundo permitiría falsificar la auditoría.
- Payer separado del firmante, para que el paciente pueda pagarse sus transacciones si el backend cae. Si se cierran cuentas, el rent vuelve al sponsor.

8 Cifrado y servicio de llaves

- **Archivo:** AES-256-GCM en el navegador, DEK aleatoria por estudio, IV único.
- **Integridad:** SHA-256 del archivo cifrado en el Record. El visor lo recalcula antes de descifrar; si no coincide, muestra "**Estudio alterado**" y no descifra.
- **Envoltura:** la DEK viaja por TLS al servicio de llaves, que la envuelve con una KEK de la organización (HKDF de una llave maestra). En el MVP, la llave maestra es un secreto de entorno; en producción, un KMS.
- **Quién recibe la DEK:** el paciente titular, el médico emisor, o un médico con AccessGrant activo y vigente. Nadie más.
- **Entrega:** DEK y URL firmada de 60 segundos. Opcional: sellar la DEK con una clave efímera X25519 del navegador del médico.
- **Auditoría:** cada entrega llama a `log_access`, más una tabla `key_releases` en Postgres.
- **Borrado:** destruir la DEK envuelta y borrar el archivo ("borrado criptográfico"). En la cadena queda solo un hash.

Origen y emisor en el visor (caso Pepito): el visor muestra quién cargó el estudio (nombre, matrícula, especialidad), la fecha on-chain y el link a la transacción. Además indica el origen: "**emitido por <centro>**" si lo cargó un médico del centro que hizo el estudio, o "**copia digitalizada por <médico>**" si lo cargó otro médico. Así quien lee puede desconfiar, por ejemplo, de una radiografía cargada por un médico de cabecera.

Qué garantiza y qué no: garantiza quién emitió cada estudio, cuándo y que nadie lo alteró después. No garantiza que el contenido sea verdadero: un médico verificado puede cargar un estudio falso desde el origen, pero el fraude queda firmado con su matrícula. La firma de wallet no es firma digital legal (Ley 25.506).

9 Backend

Módulos de NestJS: `auth` (guard que verifica el token de Supabase Auth), `organizations`, `records`, `access`, `keys`, `tx` (arma transacciones y fee payer) e `indexer`.

Endpoint	Quién	Para qué
POST <code>/records/upload-url</code>	Médico	URL firmada para subir el archivo cifrado
POST <code>/records</code>	Médico	Guarda la DEK envuelta y los metadatos; devuelve la transacción <code>issue_record</code>
GET <code>/patients/me/records</code>	Paciente	Lista de estudios y estados
POST <code>/access-requests</code>	Médico	Pide acceso con el código del paciente
GET <code>/access-requests/mine</code>	Paciente	Solicitudes pendientes
POST <code>/keys/release</code>	Paciente o médico	Entrega DEK y URL firmada si corresponde
GET <code>/audit/:recordId</code>	Paciente	Línea de tiempo de accesos
POST <code>/admin/providers/:id/verify</code>	Admin	Verifica clínica o médico

Tablas: `users` (sin DNI), `organizations`, `staff_members`, `doctors`, `records`, `access_requests`, `audit_events`, `key_releases`.

Aislamiento: `organization_id` en cada fila con RLS, rutas de almacenamiento y llaves envolventes por organización. Supabase Auth emite el token; el backend toma solo el `sub` y lee rol y organización de la tabla `app_user`, nunca del token. Para leer, crea un cliente por request con el token del usuario, así RLS decide qué filas ve. Solo el backend escribe.

10 Frontend

Vista	Qué incluye
Paciente (PWA)	Estudios, QR, marcar "no es mío", solicitudes de acceso, permisos activos y revocación, línea de tiempo.
Médico	Escáner de QR y carga de estudios; solicitar acceso; visor con marca de agua y tiempo restante.

Vista	Qué incluye
Clínica	Avalar médicos.
Admin	Verificar clínicas y médicos (puede ser un script).

Librerías: Supabase Auth, @privy-io/react-auth, TanStack Query, React Hook Form + Zod, qrcode y @yudiel/react-qr-scanner, pdfjs-dist y @serwist/next con Web Push.

Prototipo navegable listo NUEVO

Ya existe un prototipo sin backend con las vistas de paciente y médicos, en versión escritorio y celular, que recorre los pasos de la demo y muestra en vivo lo que queda en Solana. Sirve como guía visual para hcd_app y como respaldo para el pitch.

11 Estructura de repositorios

Cuatro repos en la cuenta pmpeloc. Los tres de código se clonan **dentro** de hcd, que los ignora en su .gitignore.

hcd/	github.com/pmpeloc/hcd	Documentación (docs/ en español), reglas y hooks
├ hcd_api/	github.com/pmpeloc/hcd_api	Backend NestJS + programa Anchor
└ programs/hcd/	Programa Anchor (Rust) + tests	
└ idl/	IDL publicado, del que hcd_app genera su cliente	
└ src/	API NestJS	
├ hcd_app/	github.com/pmpeloc/hcd_app	Next.js: paciente, médico, clínica y admin
└ hcd_landing/	github.com/pmpeloc/hcd_landing	Next.js estático: landing

- **Cliente del programa:** hcd_api publica el IDL en idl/. Cada repo que lo necesita genera su cliente con Codama a partir de ese archivo.
- **Esquemas Zod compartidos:** sin monorepo no hay paquete compartido. Para el MVP, los esquemas se copian de hcd_api a hcd_app.
- **Documentación y hooks:** todos los repos usan los hooks de hcd/.githubhooks y documentan en hcd/docs/. Código y commits en inglés; docs/ en español.

12 Riesgos principales

Riesgo	Mitigación
Estudio vinculado a la persona equivocada	QR presente + "no es mío"
Paciente sin celular	Fuera del MVP
Pérdida de cuenta	Recuperación de Privy
Médico falso	Verificación manual de matrícula
Servicio de llaves comprometido	Keypair propio, separado del fee payer; hoja de ruta con Arcium o Lit
Copia de lo ya descifrado	Marca de agua, sin botón de descarga, auditoría

Riesgo	Mitigación
Abuso del fee payer	Rate limit, tope diario, hot wallet con poco saldo
Spam de solicitudes	Solicitudes off-chain con rate limit
Caída de devnet	RPC de respaldo y video grabado
Bug de permisos	Tests negativos y revisión cruzada

13 Plan de 10 días

Días	Solana	Backend	Frontend	Producto y pitch
1-2 3 y 4/10	Cuentas, reglas, esqueleto, config, prestadores	Repos, base, auth con Supabase + wallet Privy (prueba de 1 hora primero)	Next.js, login, layouts por rol	Alcance, flujo y guion
3-4 5 y 6/10	issue, dispute, void, grant, revoke, log_access con tests negativos. Deploy en devnet	Cliente Codama, módulos tx y fee payer	QR y escáner, perfil	Datos de prueba
5-6 7 y 8/10	Revisión cruzada	Servicio de llaves, storage, indexer	Carga y cifrado, lista de estudios	Diapositivas del pitch
7-8 9 y 10/10	Soporte a integración	Solicitudes, notificaciones, aislamiento	Solicitud, aprobación, visor, línea de tiempo	Probar el recorrido
9 11/10	Congelar el programa	Rate limits, logs, deploy	Pulido, PWA	Video de respaldo
10 12/10	Buffer	Buffer	Buffer	Entrega, README y repo ordenado

Definición de terminado del MVP

- Un paciente se registra y un médico le carga un estudio.
- Un médico pide acceso, el paciente aprueba por 1 hora y el médico ve el estudio.
- Después del vencimiento, el servicio de llaves le niega.
- Un segundo médico sin aprobación es bloqueado.
- La línea de tiempo muestra quién accedió y cuándo, con firma verificable en un explorador.
- Dos clínicas distintas no ven los datos de la otra.
- Un archivo alterado en el storage se detecta como "Estudio alterado".

14 Demo (4:30 minutos)

Minuto	Qué se muestra
0:00	El problema
0:30	El médico carga un estudio con el QR del paciente
1:30	Otro médico pide acceso y el paciente aprueba por 1 hora
2:30	El paciente niega a un segundo médico y vence el permiso del primero
3:15	Línea de tiempo verificable y diferenciación
4:00	Límites y hoja de ruta

Hipótesis a validar: ¿quién paga primero, la clínica o la aseguradora? Si alguien tiene un contacto en una clínica de Jujuy, pedirle 15 minutos antes del cierre.

15 Hoja de ruta (después del MVP)

Del plan v1

- Reducir la confianza en el servicio de llaves con Arcium o Lit.
- Recuperación de cuenta con Swig, recuperación social y contactos de confianza.
- Acceso de emergencia con doble confirmación, registro inmediato y aviso al paciente.
- Interoperabilidad: exportar e importar HL7 FHIR (Patient, DiagnosticReport, Observation).
- Aseguradoras: verificar que un estudio existe sin ver su contenido, para evitar duplicados.
- Escala: importación masiva con Merkle roots, KMS por organización, auditoría externa y mainnet.
- Validación legal con un abogado especializado en datos de salud y firma digital.

Del caso Pepito (médico cómplice que carga un estudio falso)

- **Alerta de conflicto:** marcar cuando el médico que cargó un estudio es el mismo que después pide acceso o receta en base a él.
- **Suspender prestadores:** si se descubre un fraude, el admin le quita la verificación y todos sus estudios se muestran con la advertencia "emisor suspendido".
- **Auditoría como evidencia:** exportar el registro firmado de un médico para presentarlo ante el colegio médico.
- **Centros de imágenes como emisores directos**, para que sus estudios figuren como "emitido por el centro".
- **Detección de patrones:** un médico de cabecera que carga muchas imágenes, o muchos estudios para un mismo paciente.

Del relevamiento de Franco

Página para verificar copias que circulan por fuera, validar la firma digital del PDF (PAdES), multisig con Squads para la clave admin, conexión con REFEPS/SISA y cobro por organización de los lamports consumidos.

